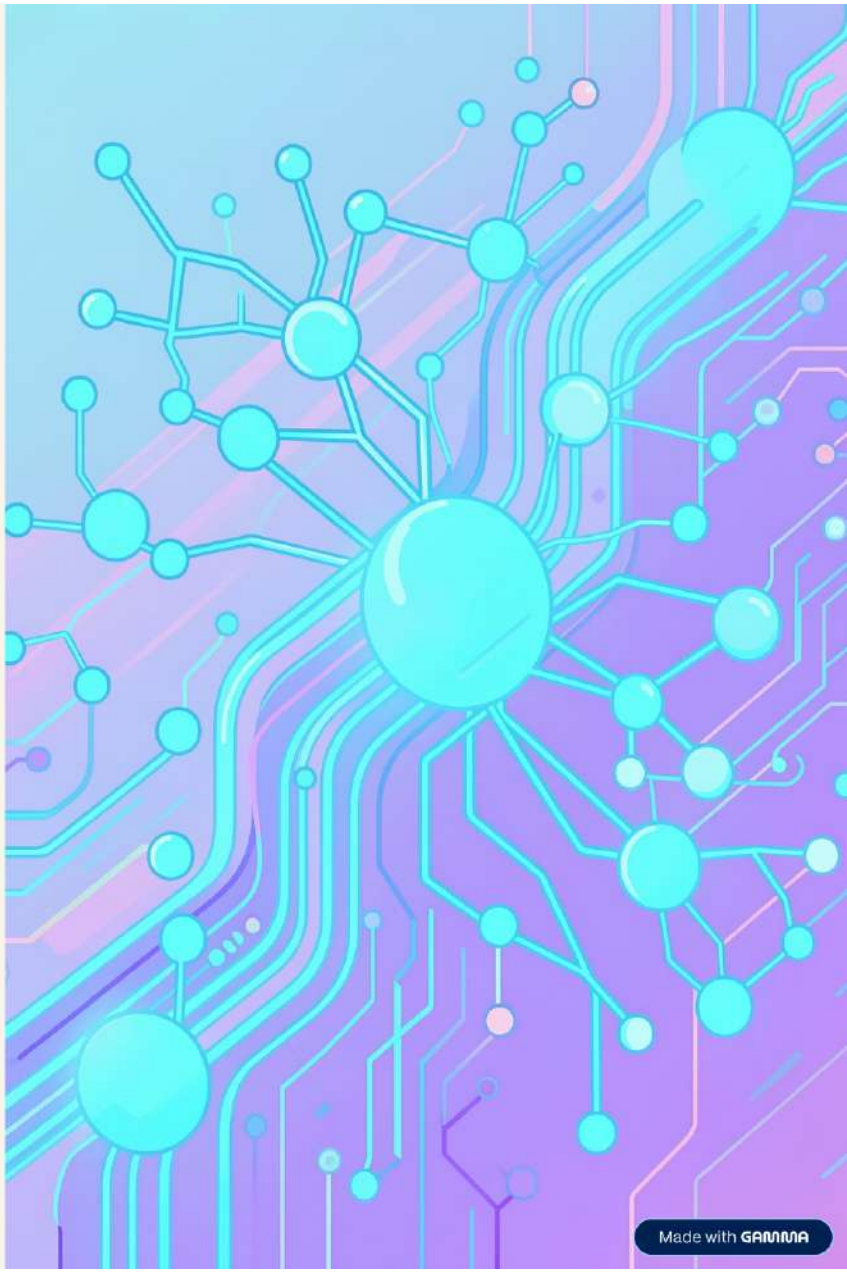
SQLite → HiveQL Translator

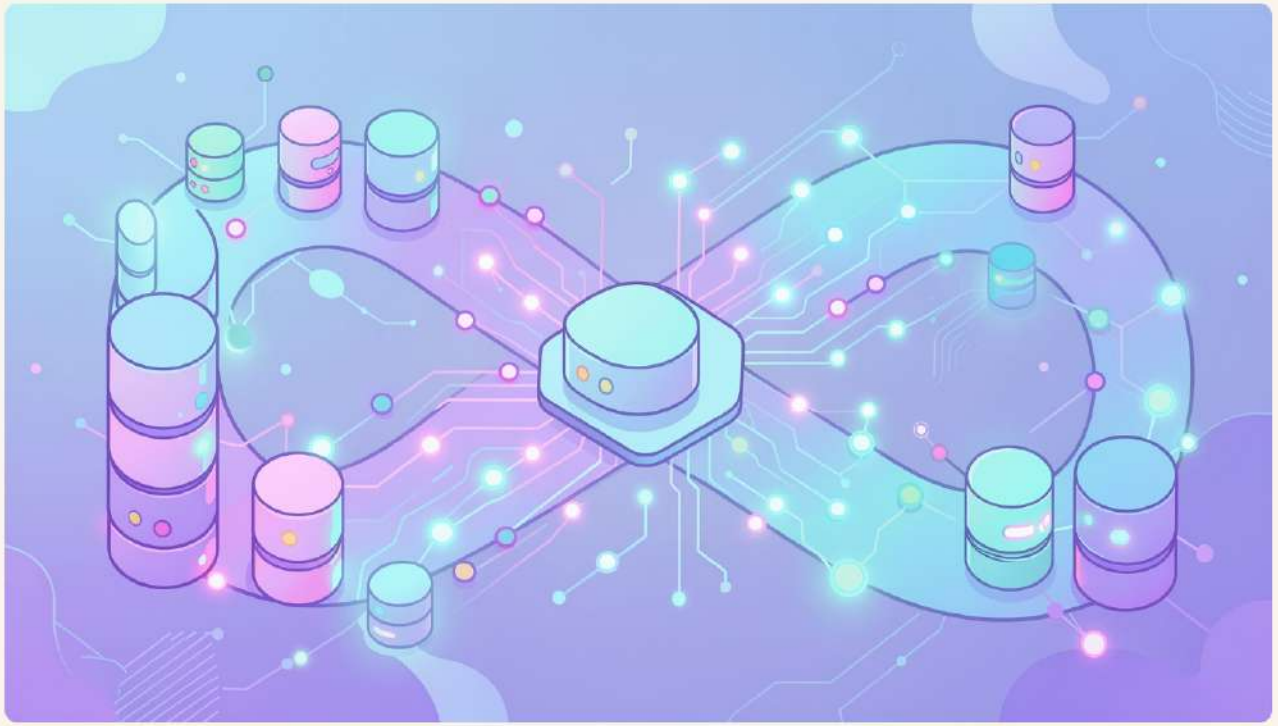
GenAI-Powered SQL Refactoring Assistant

Milestone 6: Deployment & Documentation · BSDA4001 — DS & AI Lab · IIT Madras

Team: Ajay, Madhavan, Sanjay, Senthil, Veral



The Dialect Problem



SQL is not a single language — it is a family of dialects. Translating between them manually is slow and error-prone.

Date Functions

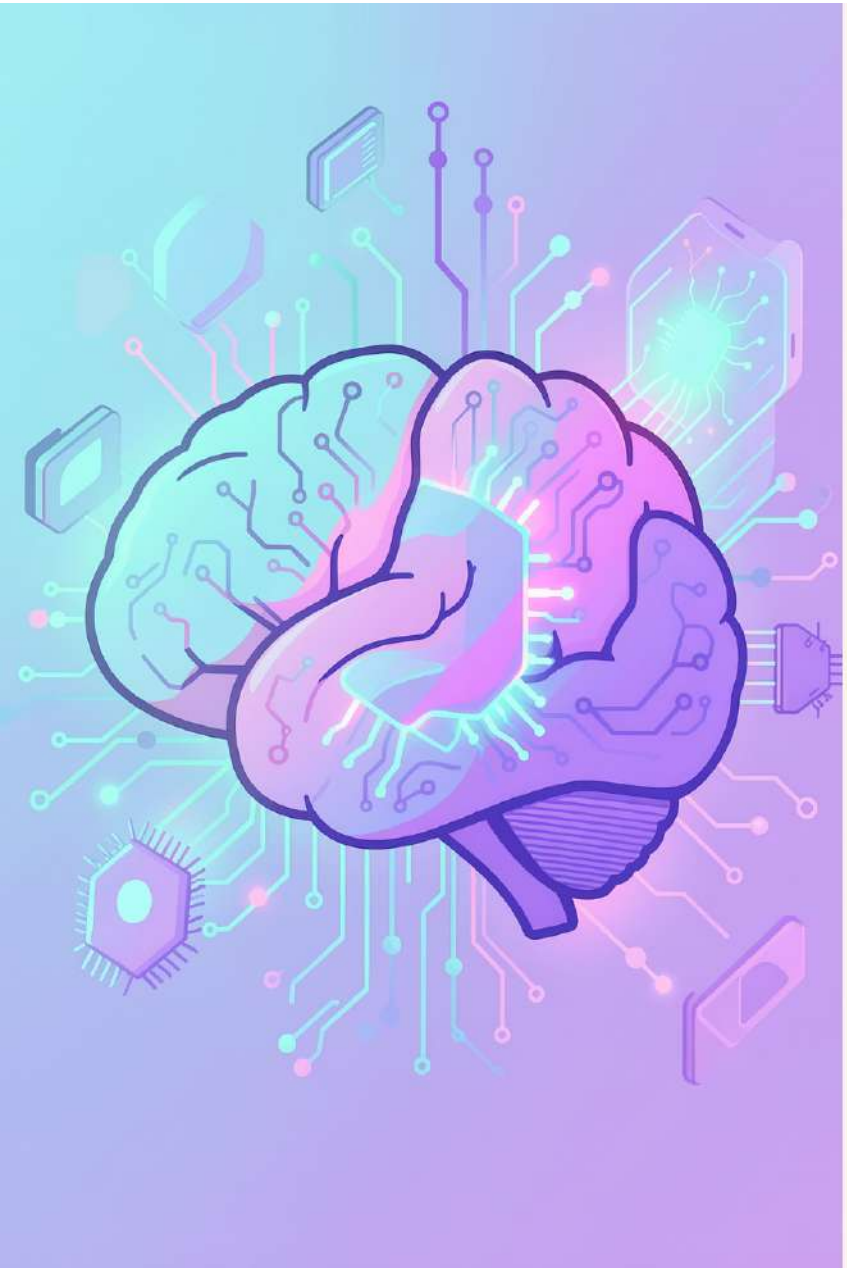
SQLite's `strftime()` vs. HiveQL's `from_unixtime()` — semantically equivalent, syntactically distinct.

Upsert Semantics

SQLite uses `INSERT OR REPLACE`; HiveQL requires `INSERT OVERWRITE` with partition logic.

Function Names & Syntax

Core operations differ in naming, argument order, and type coercion rules across dialects.



Project Objective

🧠 Neural Translator

Fine-tune T5-base to learn SQLite → HiveQL mappings with full semantic equivalence.

🎯 High Accuracy

Retrieval-augmented generation (RAG) ensures contextually correct, production-grade output.

⚡ Real-Time Serving

Deployed as an interactive Gradio web app and a production FastAPI REST API.

MILESTONE 6 — FINAL SYSTEM

Best Configuration: Full Pipeline

The final system combines three production-grade components into a unified, end-to-end translator.

T5-base Fine-Tuned

220M-parameter seq2seq model, fully fine-tuned on 5K parallel query pairs.

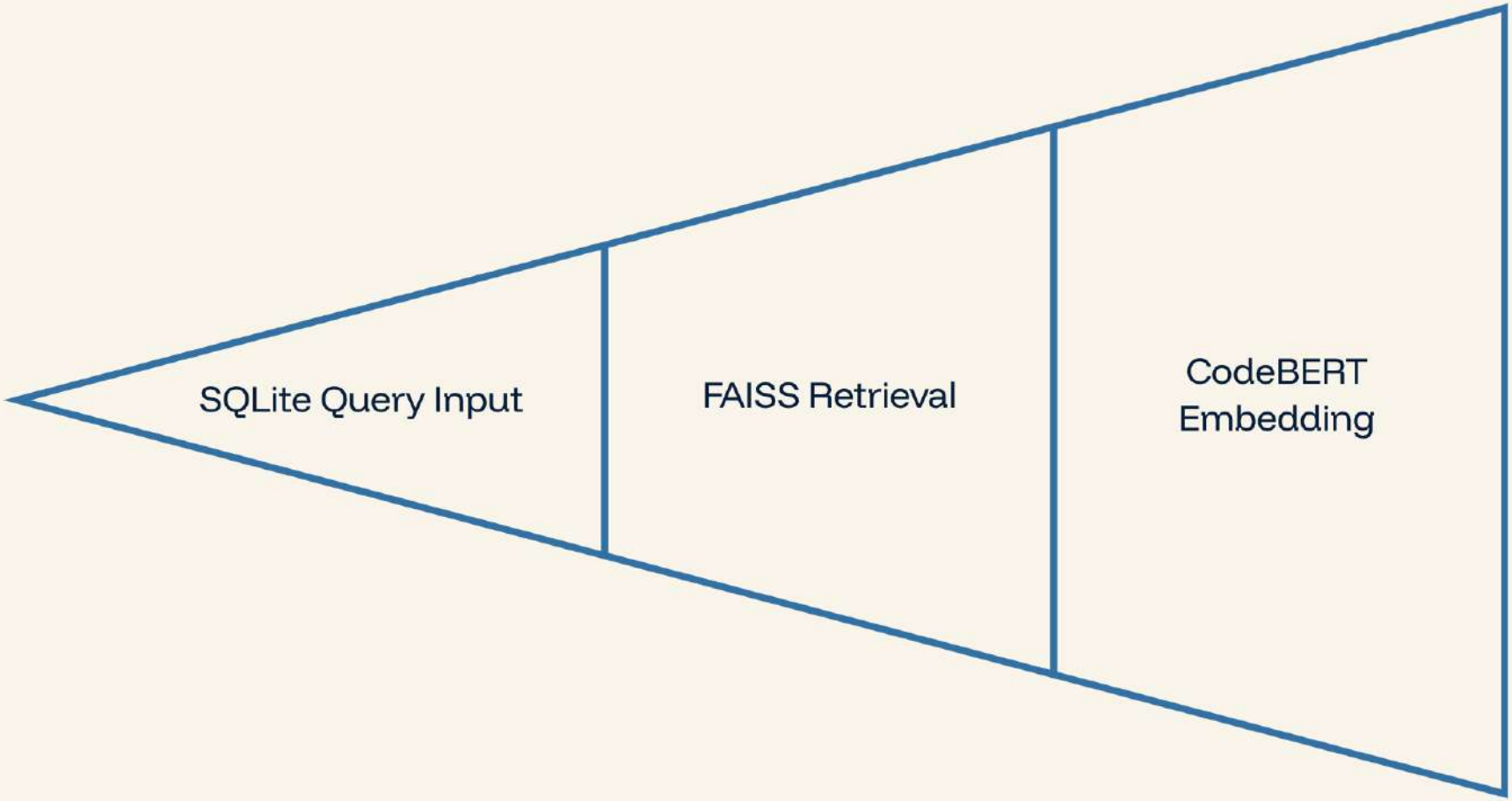
RAG Layer

CodeBERT embeddings + FAISS index for retrieval-augmented few-shot prompting.

Deployment Stack

Gradio UI for interactive testing; FastAPI for programmatic integration.

Architecture Overview



The RAG layer injects semantically similar examples into the prompt context, guiding T5 toward accurate, dialect-correct translations at inference time.

Key System Components

Translation Model

T5-base — 220M parameters. Encoder-decoder architecture fine-tuned end-to-end on SQLite–HiveQL parallel data.

Retriever Engine

CodeBERT generates dense query embeddings. **FAISS** performs approximate nearest-neighbor search for top-k retrieval.

Serving Layer

Gradio provides an interactive browser UI. **FastAPI** exposes a low-latency REST API for programmatic access.

DATA PIPELINE

Dataset Construction

No public SQLite → HiveQL parallel corpus existed. The team built a high-quality dataset from scratch.

01

Seed Queries

600 hand-crafted SQLite queries covering diverse SQL patterns and complexity levels.

02

Manual Translation

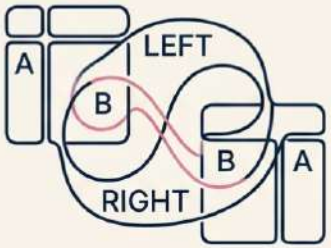
Each seed query manually translated to HiveQL, establishing ground-truth parallel pairs.

03

AST Augmentation

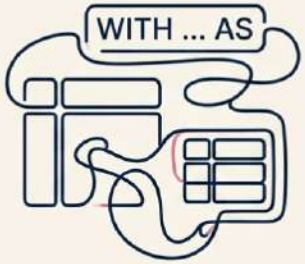
Programmatic transformations expanded the corpus to **~5,000 training pairs.**

AST-Based Augmentation



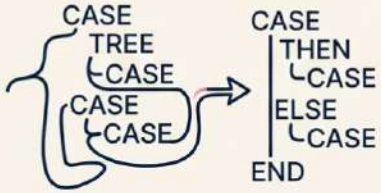
JOIN Reordering

Swap left/right join tables



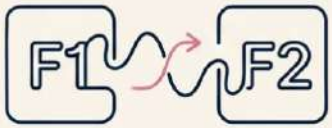
Subquery Extraction

Inline subqueries to CTEs



CASE Rewrites

Restructure conditional expressions



Function Substitutions

Swap equivalent built-in functions

Why AST, Not String Manipulation?

Abstract Syntax Tree transformations preserve **semantic equivalence** while varying surface syntax — critical for robust model generalization.

10 transformation types applied, chained up to 3×, yielding an 8× expansion from 600 seeds to ~5,000 pairs.

Data Preprocessing

Tokenization

T5Tokenizer with max sequence length of **256 tokens**, balancing context coverage and memory efficiency.

Special Tokens

Custom sentinel tokens `<sqlite>` and `<hiveql>` mark source and target dialect boundaries in the input sequence.

Schema Context

Optional table schema injection provides column-level context, improving translation accuracy for ambiguous queries.

Dataset Split & Integrity

~4K

Training Pairs

Used for T5 fine-tuning with gradient descent optimization.

~500

Validation Pairs

Used for hyperparameter tuning and early stopping.

~500

Test Pairs

Held-out evaluation set for final BLEU and accuracy reporting.

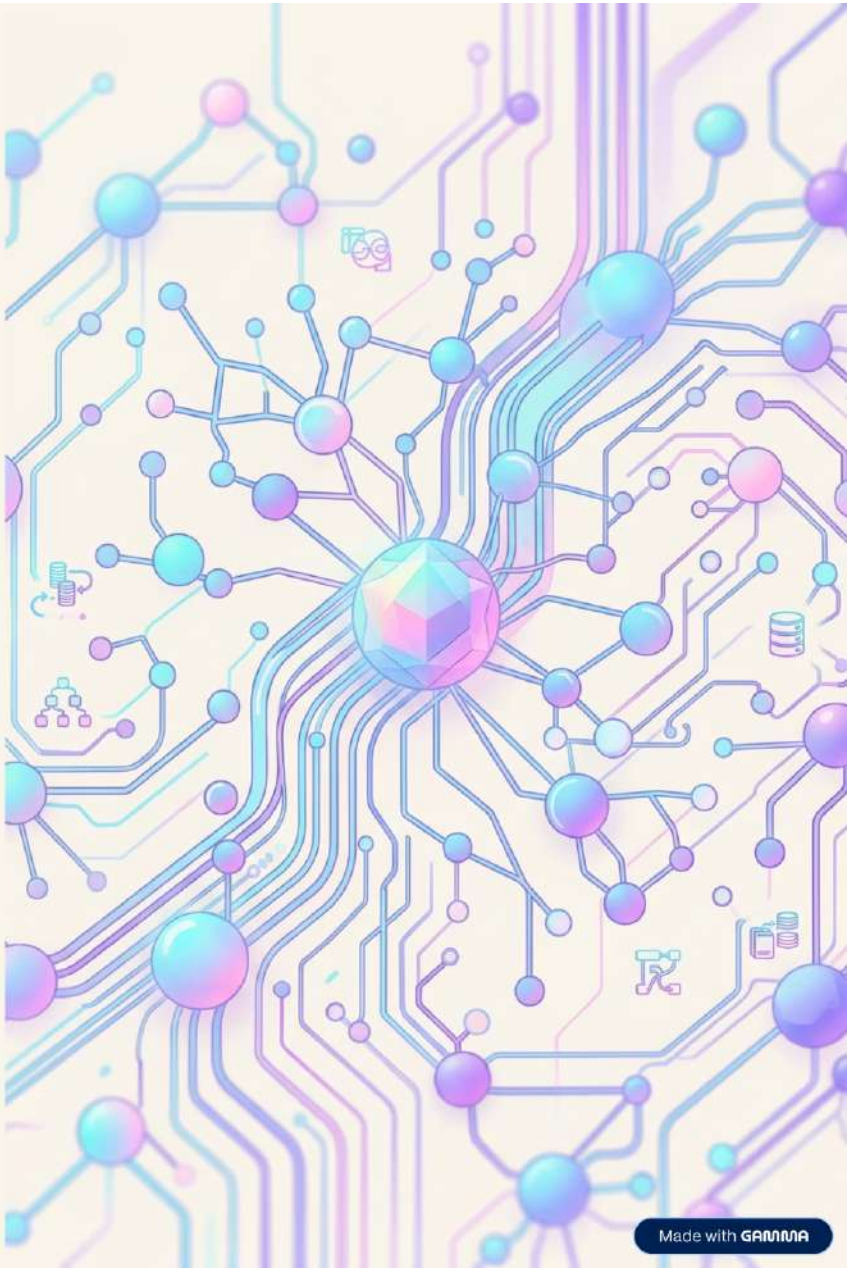
Critical design decision: Train/val/test split performed before AST augmentation to prevent data leakage — no augmented variant of a test query appears in training.

SECTION 4 — MODEL & TRAINING

Model Architecture

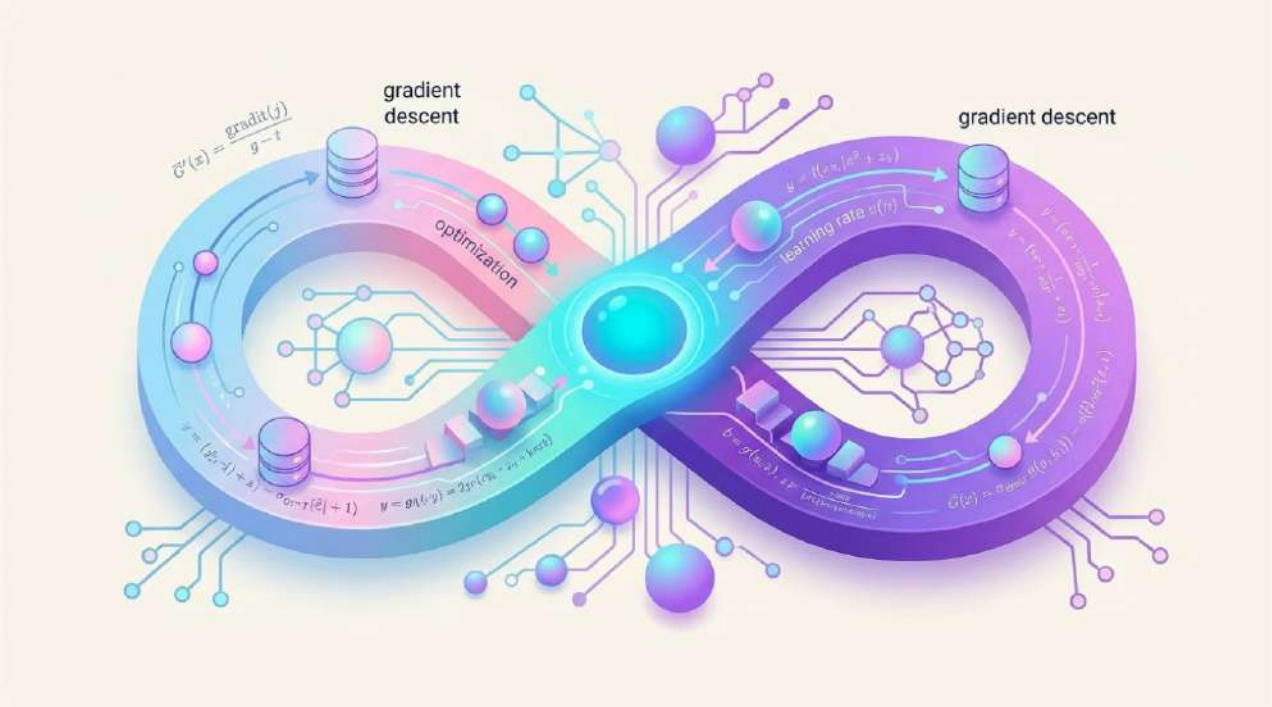
Built on **T5-base** with 220M parameters — fully fine-tuned end-to-end. No adapters, no shortcuts.

- **Base Model:** T5-base (text-to-text transfer transformer)
- **Parameters:** 220M
- **Fine-tuning:** 100% of layers trained — no frozen weights
- **No LoRA / Adapters:** Full-parameter optimization for maximum task alignment



SECTION 4 — TRAINING CONFIGURATION

Training Configuration



Optimizer

AdamW — adaptive learning rate with decoupled weight decay for stable convergence

Learning Rate

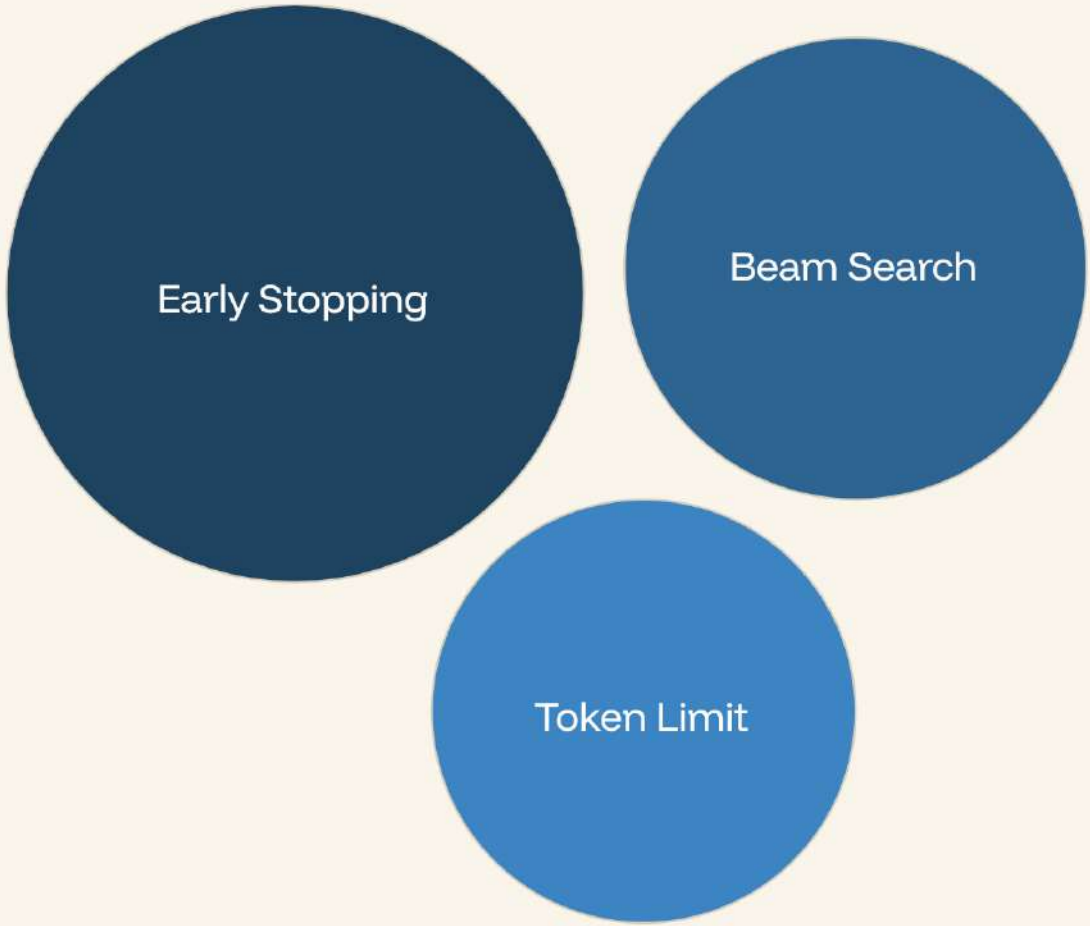
5e-4 with Warmup + Cosine decay schedule to balance exploration and convergence

Epochs & Batch

10 epochs, batch size 16 (gradient accumulation → effective batch 32)

SECTION 4 — TRAINING STRATEGY

Training Strategy



Early stopping halts training after 3 epochs without validation improvement, preventing overfitting. Beam search with `num_beams=4` explores multiple candidate sequences in parallel, selecting the highest-probability output. The 256-token ceiling ensures concise, well-bounded SQL generation.

SECTION 5 — RAG PIPELINE

Why RAG?

Retrieval-Augmented Generation injects **relevant context** into the prompt at inference time — dramatically improving translation quality for complex SQL patterns.

Context Examples

Few-shot guidance from semantically similar prior translations

Function Mapping

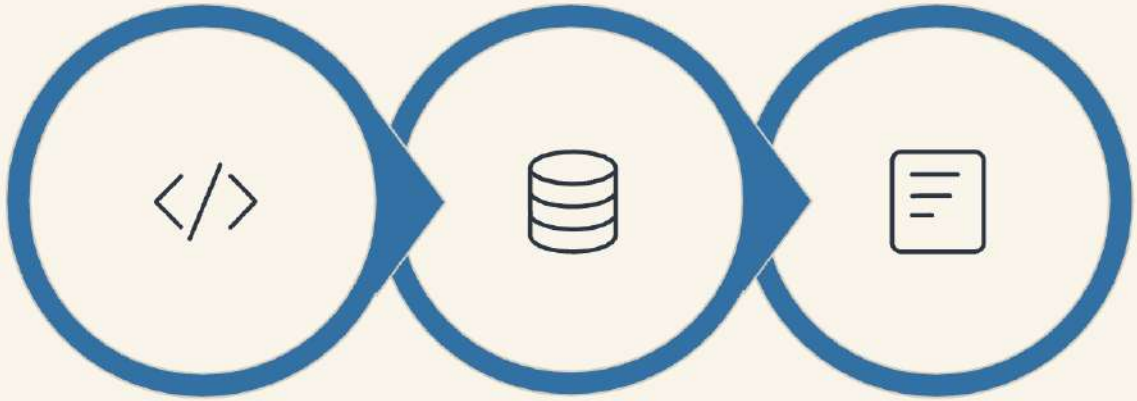
Helps model resolve dialect-specific function equivalents

Structural Transforms

Guides nested query and JOIN pattern rewrites

SECTION 5 — RAG DESIGN

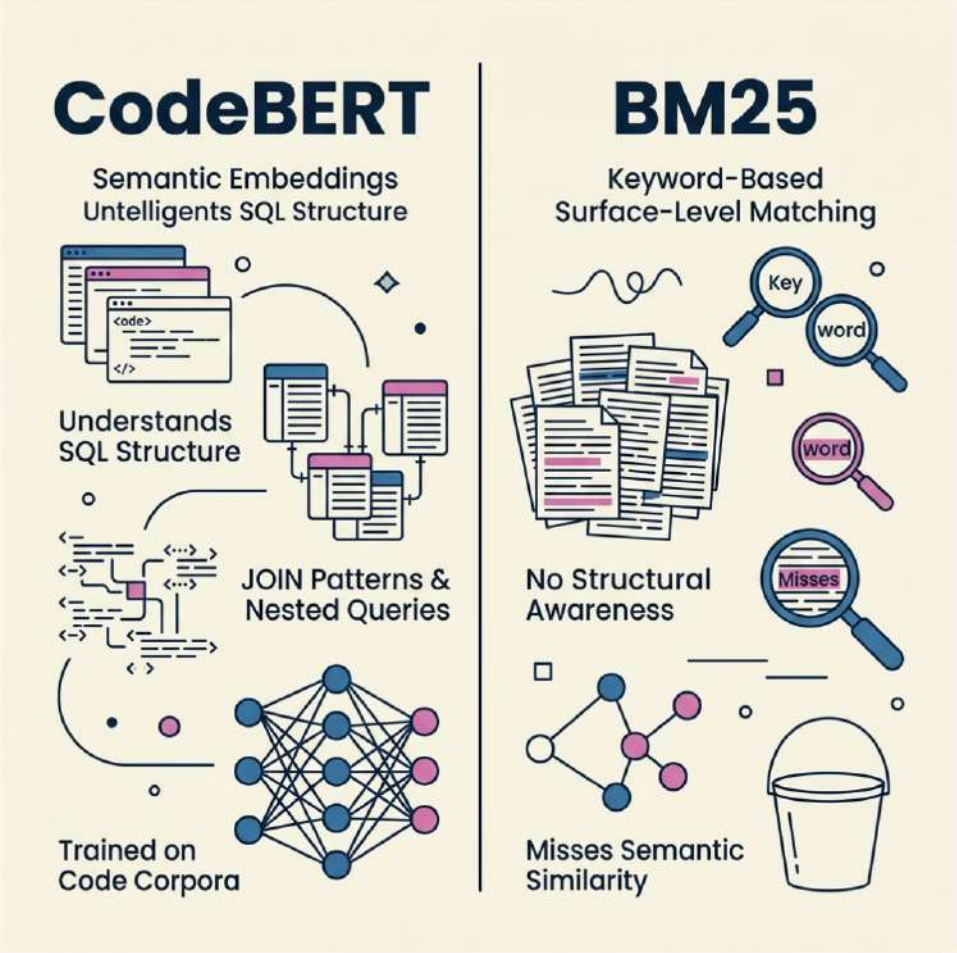
RAG Pipeline Design



The pipeline encodes the incoming SQLite query using CodeBERT, searches the FAISS index for the most semantically similar historical examples, and appends them to the prompt — enabling **few-shot learning at inference time** without retraining.

SECTION 5 — EMBEDDING MODEL

Why CodeBERT?



CodeBERT is pre-trained on code and natural language corpora, giving it a structural understanding of SQL that BM25 cannot match.

- Captures **SQL syntax structure** and JOIN patterns
- Understands **nested query** relationships
- Semantic similarity over keyword overlap
- Trained on code — domain-aligned by design

SECTION 5 — RETRIEVAL ENGINE

FAISS Retrieval

~5K

Dataset Size

Translation example pairs indexed for retrieval

<5ms

Retrieval Latency

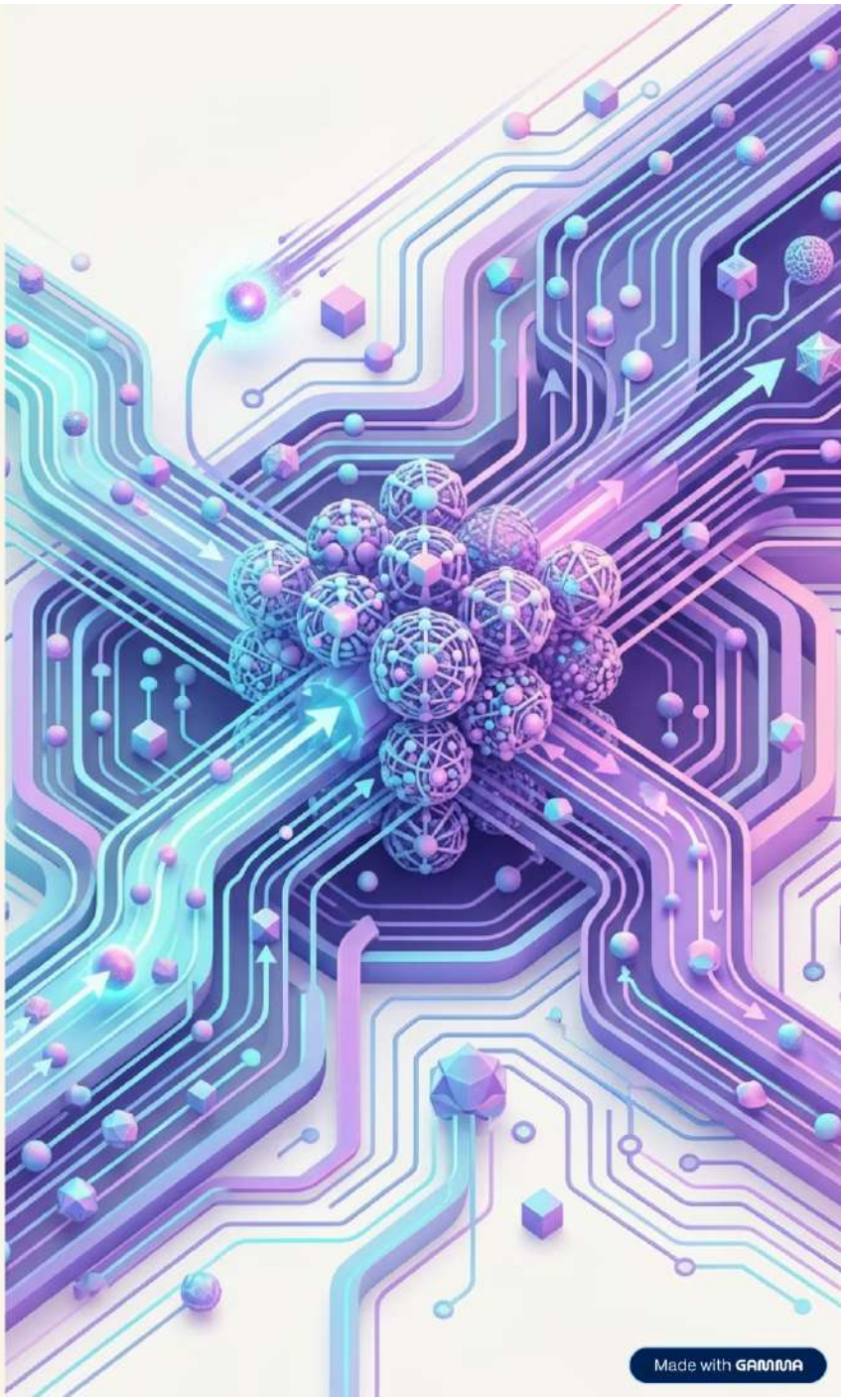
Exact L2 search via IndexFlatL2

100%

Exact Search

No approximation — full precision vector matching

IndexFlatL2 performs exact nearest-neighbor search using L2 distance — ideal for datasets under ~10K where latency and precision are both critical.



SECTION 5 — PROMPT ENGINEERING

Prompt Construction

Prompt Format

```
translate sqlite to hiveql:

<example1>
<example2>
<example3>

<query>
```

Retrieved examples are injected before the target query, giving the model **contextual guidance** through in-context learning.

Why This Works

- Examples demonstrate the **target dialect pattern**
- T5 attends to retrieved context via self-attention
- No additional training — pure inference-time adaptation
- Top-k selection ensures **relevant** demonstrations

SECTION 6 — INFERENCE PIPELINE

Inference Flow

01

Receive Query

Raw SQLite query from user or upstream system

02

Encode with CodeBERT

Generate dense vector embedding for semantic retrieval

03

Retrieve Top-k Examples

FAISS search returns most similar historical translations

04

Build Prompt

Inject examples into structured few-shot template

05

Generate with T5

Beam search decoding produces final HiveQL output

SECTION 6 — CORE LOGIC

Core Inference Function

```
if use_rag:
    emb = codebert_encode(query)
    examples = faiss.search(emb)
    prompt = build_prompt(query, examples)
else:
    prompt = direct_prompt(query)

output = t5_generate(prompt)
```

Design Highlights

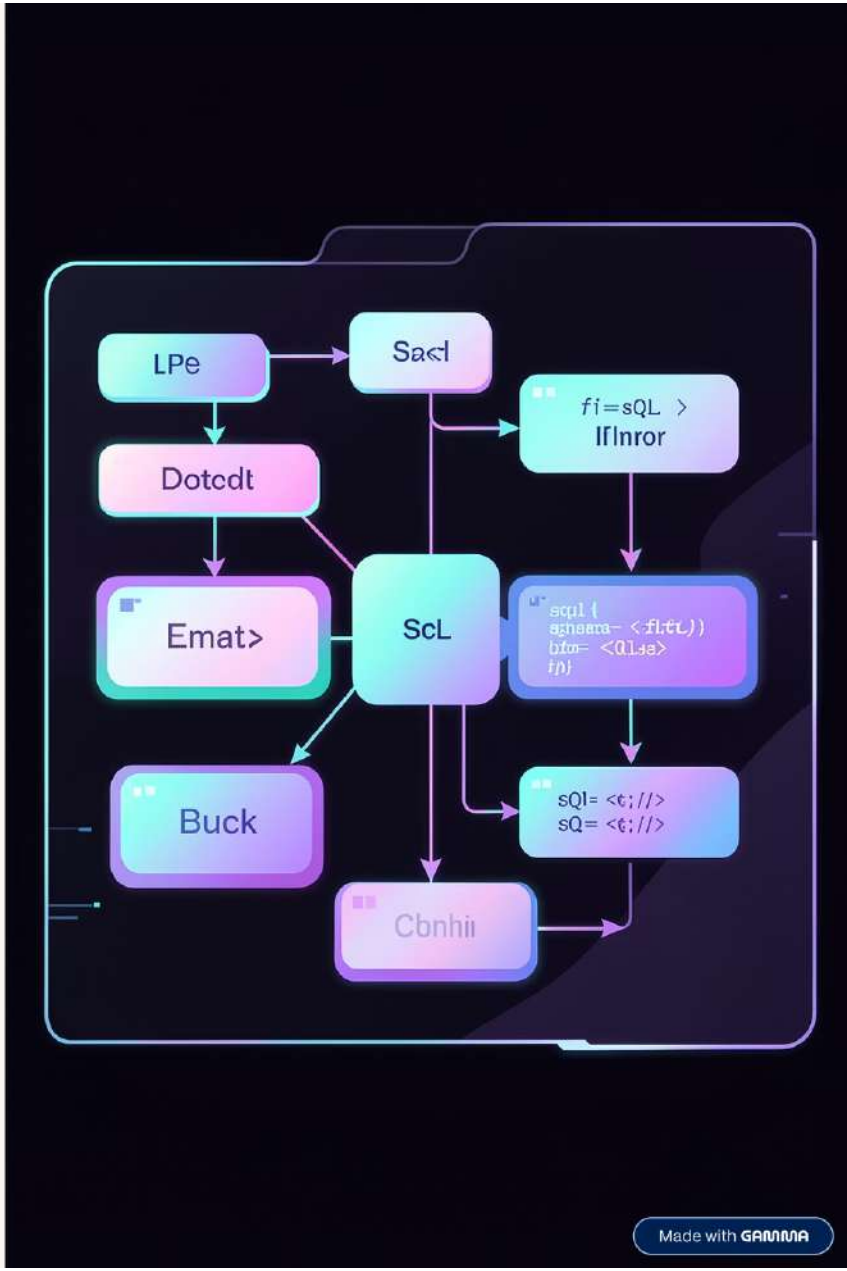
- **RAG toggle** enables A/B testing and fallback
- Modular: encoder, retriever, and generator are **swappable**
- Single `t5_generate()` call handles all output logic
- Latency-critical path: encode + retrieve under **10ms** total

RAG path adds ~5ms overhead for measurable quality gains on complex queries.

SYSTEM INTERFACE

Input / Output Format

The translation system accepts standard SQLite queries and optional retrieval parameters, returning a fully formed HiveQL query alongside retrieval metadata and latency measurements.



What Goes In, What Comes Out

Input

- SQLite query (required)
- `use_rag` flag (optional)
- `top_k` retrieval count (optional)

Output

- Translated HiveQL query
- Retrieved few-shot examples (when RAG enabled)
- Inference latency measurement

This clean contract enables both programmatic API access and interactive UI usage without friction.

SECTION 7 — RESULTS & EVALUATION

Evaluation Metrics

Three complementary metrics assess translation quality across surface form, exact correctness, and character-level fidelity.

BLEU

Primary metric. Measures n-gram overlap between predicted and reference HiveQL, capturing fluency and structural similarity.

Exact Match

Percentage of predictions that match the reference query character-for-character — the strictest correctness signal.

chrF

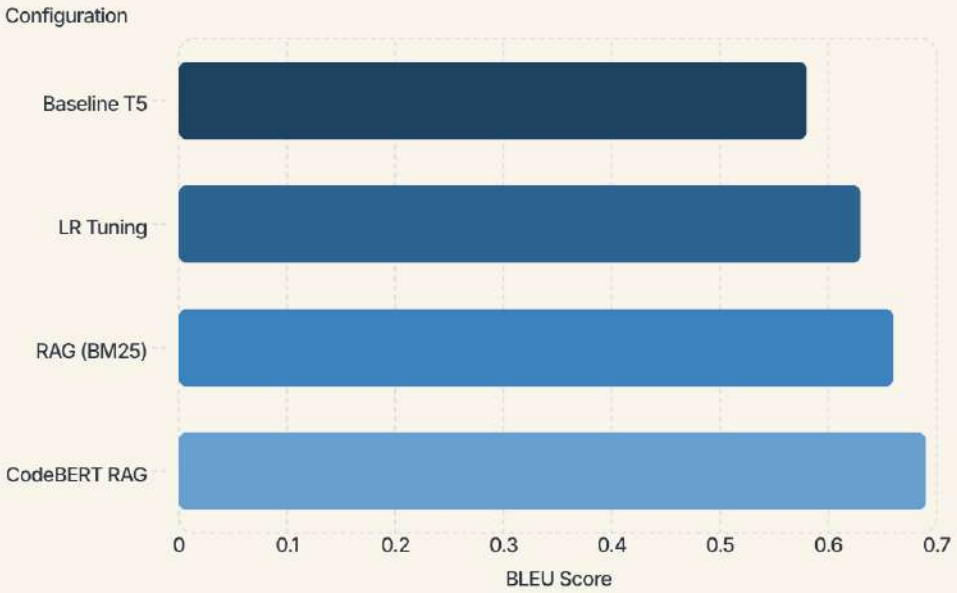
Character-level F-score that is more robust to minor tokenization differences, especially useful for SQL syntax.

Final Results — Best Configuration

0.69	33%	0.65
BLEU Score	Exact Match	chrF Score
Best across all configurations	Queries perfectly reproduced	Character-level fidelity

All three metrics show significant improvement over the baseline T5 model, validating the RAG-enhanced approach.

Performance Comparison Across Configurations



RAG Adds Real Value

Each architectural improvement compounds on the last. The jump from baseline (0.58) to CodeBERT RAG (0.69) represents a **19% relative gain** in BLEU — driven primarily by structural retrieval quality.

→ LR tuning adds +0.05 BLEU

→ BM25 RAG adds +0.08 BLEU

→ CodeBERT RAG adds +0.11 BLEU

KEY INSIGHTS

What the Results Tell Us

Structural Retrieval Wins

CodeBERT embeddings outperform BM25 keyword search — capturing query structure matters more than surface tokens.

k=3 is Optimal

Three retrieved examples balance context richness against noise; higher k degrades performance.

Full Fine-Tuning Works

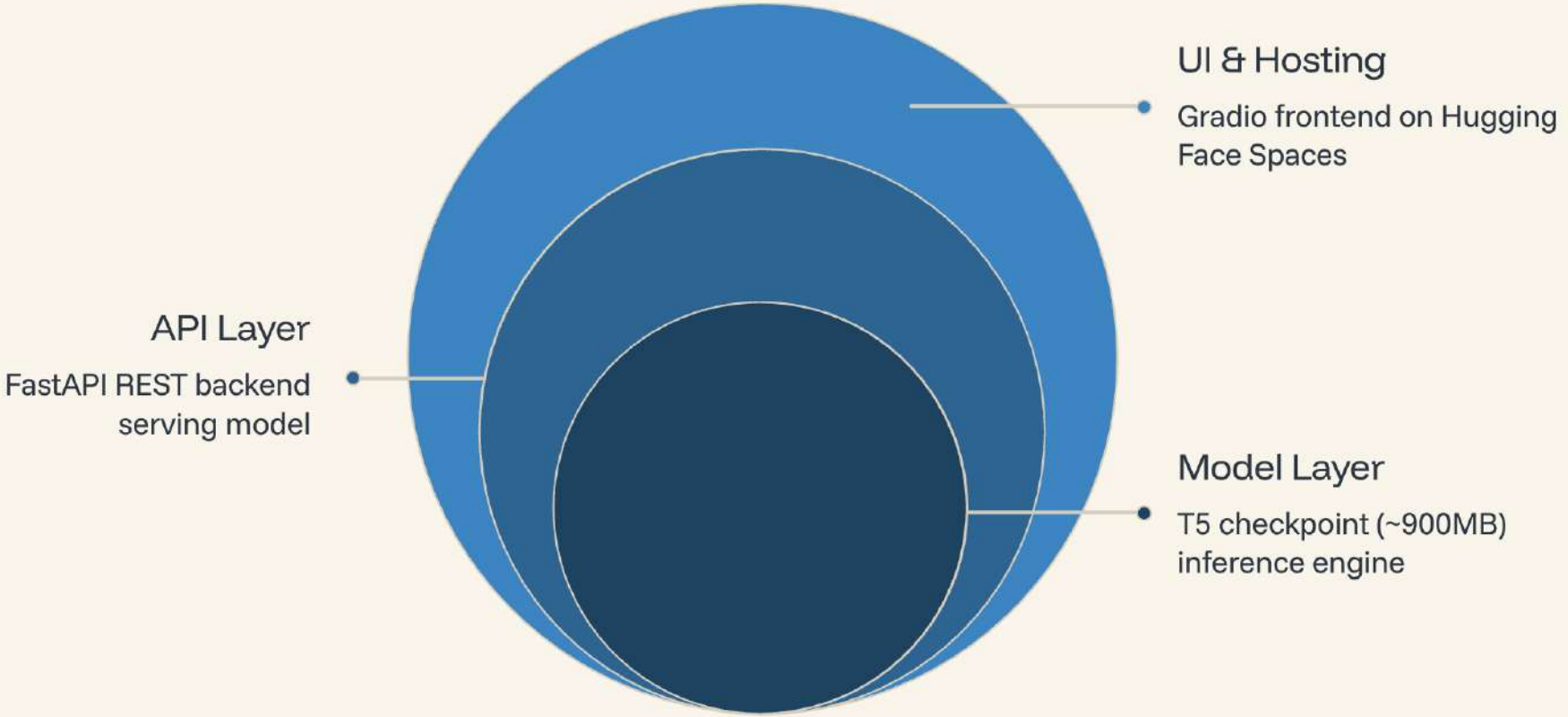
Full parameter fine-tuning outperforms parameter-efficient alternatives for this task size.

RAG Improves Generalization

Retrieval-augmented inference helps the model handle out-of-distribution query patterns more robustly.

SECTION 8 — DEPLOYMENT

Deployment Architecture



The system is containerized and deployed on Hugging Face Spaces, with a lightweight FastAPI backend serving the ~900MB T5 checkpoint and a Gradio frontend providing an accessible UI for interactive translation.

User Flow & API Endpoints

Interactive User Flow

01

Enter SQLite query in the text field

02

Toggle RAG retrieval (optional)

03

Click **Translate**

04

View HiveQL output + retrieved examples

REST API

POST /translate

Accepts `query`, `use_rag`, `top_k`. Returns HiveQL + metadata.

GET /health

Liveness check for monitoring and orchestration.

```
curl -X POST /translate \
 -H "Content-Type: application/json" \
 -d '{"query": "SELECT * FROM users"}'
```

Inference Performance

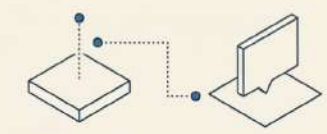
1–3s	~25s	T4
Typical Latency	Cold Start	GPU Hardware
End-to-end translation time under normal load	Model warm-up time on first request after idle	NVIDIA T4 GPU on Hugging Face Spaces

Cold start latency is a known constraint of serverless GPU hosting. A persistent warm instance eliminates this overhead for production deployments.

SECTION 9 — SYSTEM DESIGN

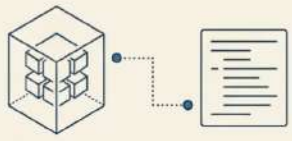
Key Design Decisions

Each architectural choice prioritized simplicity, speed, or translation quality — with clear trade-offs documented.



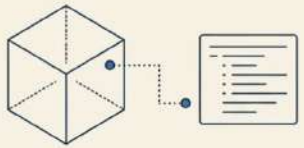
Direct Embedding **HyDE**

Direct chosen for faster inference with no generation overhead



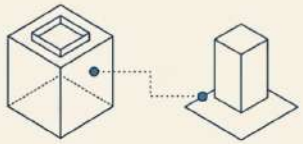
CodeBERT **BM25**

CodeBERT chosen for superior structural query matching



Direct Embedding **BM25**

Direct chosen for superior structural query generation overhead



Full Fine-Tuning **LoRA**

Full FT chosen for simpler deployment and better end-to-end performance

Why These Choices Matter

The system balances retrieval quality against inference latency at every layer. CodeBERT's structural embeddings and full fine-tuning together deliver the strongest generalization without introducing serving complexity.

Speed

Direct embeddings

Quality

CodeBERT RAG

Simplicity

Full FT deploy

ERROR HANDLING

Robust Error Handling

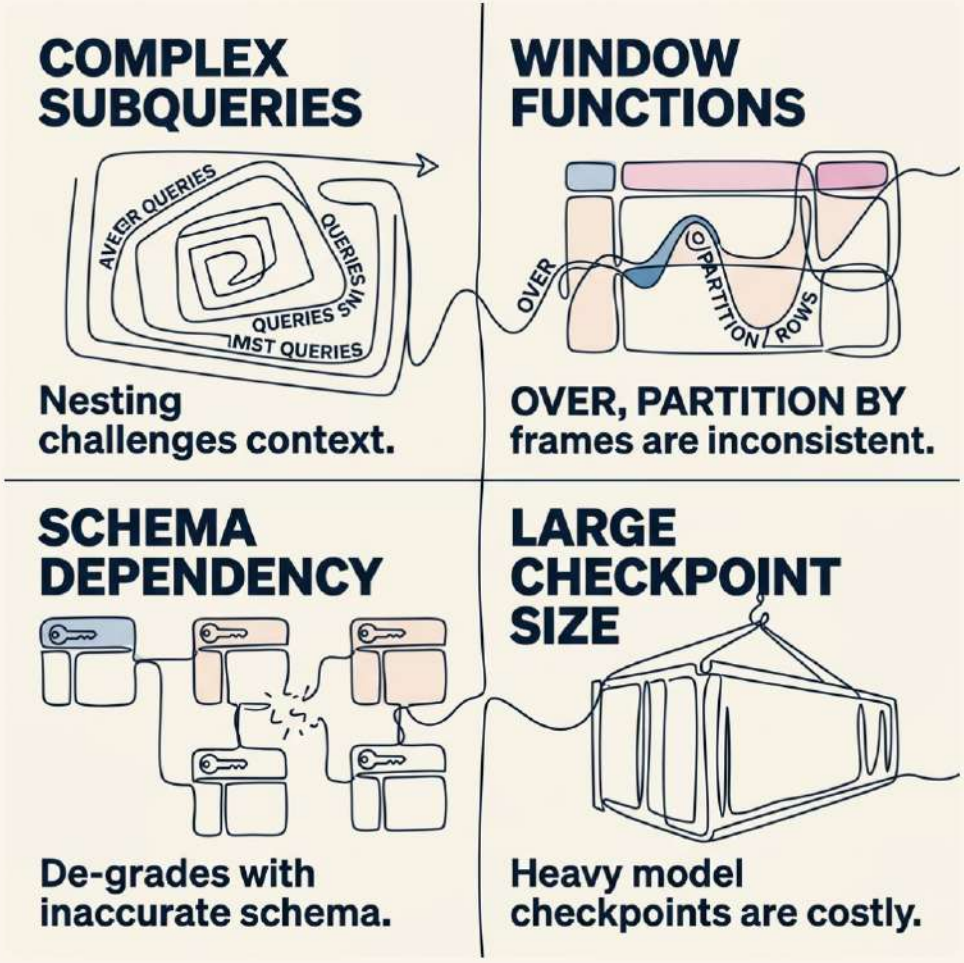
Four layers of resilience keep the system stable and traceable under real-world conditions.

- **SQL Validation:** `sqlglot` parses and validates every generated query before execution
- **Timeout Handling:** Prevents runaway inference from blocking the pipeline
- **RAG Fallback:** When generation fails, retrieval augments or recovers the output
- **Logging:** Structured logs capture every step for debugging and auditability

SECTION 10 — LIMITATIONS

Known Limitations

Four areas where the current architecture faces constraints.



These limitations are acknowledged trade-offs in the current architecture and represent direct targets for the next iteration.

SECTION 11 — FUTURE WORK

Future Improvements

More SQL Dialects

Extend support beyond current targets to cover additional database flavors

Schema-Aware Translation

Inject live schema metadata at inference time for higher accuracy

LoRA Fine-Tuning

Low-Rank Adaptation for lightweight, efficient deployment

Larger Models

Upgrade to CodeT5+ for improved code generation quality

Active Learning

Continuously improve from production feedback and edge cases

SECTION 12 — CONCLUSION

Production-Ready SQL Translation

An end-to-end GenAI system combining fine-tuning, RAG, and deployment — validated with real metrics.

0.69

BLEU Score

Translation quality on held-out evaluation data

<1s

Real-Time Inference

Latency suitable for interactive workloads

3

Core Techniques

Fine-tuning, RAG, and deployment unified in one pipeline

This system is production-ready — a fully operational GenAI pipeline for SQL translation.